université de BORDEAUX

# Automatic tuning of the IO500 benchmark parameters with Bayesian Optimization

M2 Computer Science
Internship Report

Inria Centre at the University of Bordeaux

**[TODO:** student name**]**
Supervisors: **[TODO:** supervisor names**]**

August 14, 2026

# Contents

**1. Introduction** **1**
1.1. Objectives of the internship . . . . . 3
1.2. Contributions . . . . . 3
1.3. Organisation of the report . . . . . 4

**2. Context of the internship** **4**

**Bibliography** **6**

# 1. Introduction

High Performance Computing (HPC) systems are used to run large-scale scientific and engineering applications. ~~These applications often process very large amounts of data and use many computing resources in parallel.~~ The applications use many compute resources in parallel and often process large amounts of data. Therefore, ~~their performance~~ an application's performance does not only depend on the computing power of the system, but also on the performance of its storage infrastructure. ~~The ability to access and store data efficiently is also an important factor.~~ In many HPC workloads, I/O operations ~~can become a bottleneck and limit the~~ become the bottleneck that limits overall application performance.

~~This problem is becoming more important as the amount of data produced and processed by HPC applications continues to increase. The growth of data can be faster than the ability of storage systems to absorb and process it. Improving I/O performance is therefore an important research and engineering problem in HPC.~~

Parallel file systems (PFS) such as Lustre [6], BeeGFS [11] and GPFS [5] are commonly used to provide high-performance storage in HPC environments. In a PFS, the storage capacity is exposed as a set of logical units called Object Storage Targets (OSTs). When a file is written, it is split into fixed-size chunks that are sent in parallel to several targets, so that the bandwidth of many storage devices is used at once. This process is known as striping and is illustrated in Figure 1. How many targets a file is spread over, called the stripe count, and how large each chunk is, called the stripe size, are configuration choices rather than fixed properties of the system. Therefore, ~~These systems~~ a PFS exposes configuration parameters that can affect I/O performance. ~~For example~~ Beyond striping, performance can also depend on ~~the striping configuration, the number of storage targets used,~~ the transfer size, the number of processes, or the placement of processes on compute nodes. The best values for these parameters are not necessarily the same for every machine: a configuration that performs well on one system may perform poorly on another one.

To evaluate the performance of these systems, we rely on benchmarks. A benchmark is a program designed to stress a specific part of a system under controlled and reproducible conditions, so that the measurements it produces can be compared across machines and configurations [2]. For I/O, two of the most widely used benchmarks are IOR [8], which measures the bandwidth reached by large parallel read and write operations, and mdtest [9], which measures the rate at which metadata operations such as creating, opening and removing files are performed. A different approach is taken by mini-applications, or miniapps [1], which reproduce the access pattern of a real scientific application in a reduced and self-contained form, and therefore measure performance under a workload closer to production use. Each of these benchmarks characterises one aspect of the system, and none of them alone describes the storage system as a whole.

~~This~~ Thus, the variety of parameters and access patterns makes the evaluation of I/O performance difficult~~. A~~ : a benchmark must not only measure the performance of the storage system, but ~~should~~ also provide a meaningful way to compare different systems and configurations. For this reason, the HPC community maintains rankings in which every system runs the same benchmarks under a common set of rules, so that the resulting scores can be compared and ordered. The best known of them is the TOP500 [15], which ranks HPC platforms by their computational performance. In the I/O world, the community created the IO500 benchmark suite [3]. ~~was designed for this purpose.~~ IO500 is widely used to evaluate and rank the I/O performance of HPC storage systems, and it

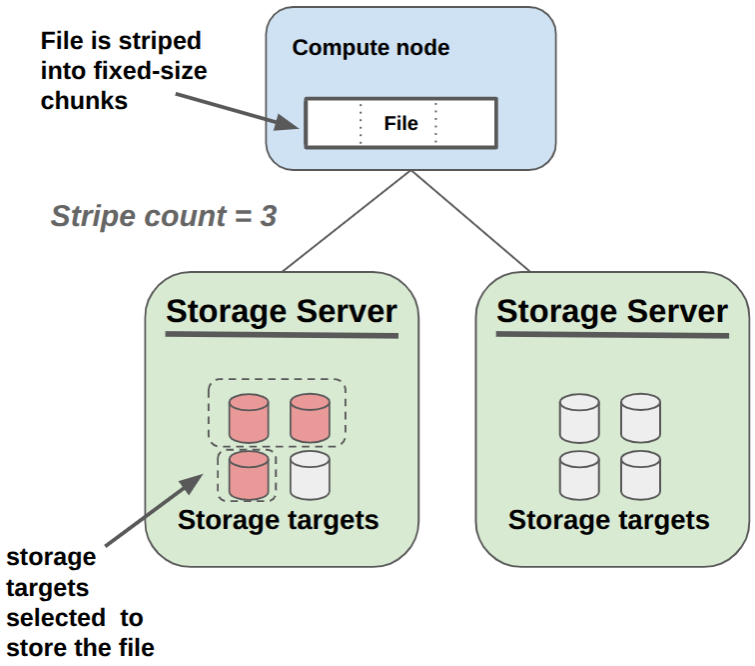



Figure 1: Striping in a parallel file system. A file created on a compute node is split into fixed-size chunks, which are written in parallel to storage targets hosted by the storage servers. Here the stripe count is three, so the file is spread over three of the eight available targets.

combines several workloads, including bandwidth-oriented and metadata-oriented operations, into a single score~~The benchmark results are also~~ , which is used to produce the IO500 ranking.~~, which is published twice a year.~~

Because IO500 is a ranking, there is a direct incentive for HPC sites to obtain the best possible score. For vendors and system integrators, a high position is a commercial argument, used in marketing material and in procurement discussions. For computing centres, it is a public and comparable demonstration that the machine they operate delivers the capability it was built for, which matters both for the visibility of the centre and for justifying the investment it represents. ~~The~~ However, the reported performance of a system ~~therefore~~ does not only depend on its hardware~~. It also depends~~ , but also on how the system is configured and how the benchmark is executed. ~~Finding a good configuration can consequently have a significant impact on the measured IO500 score.~~ Finding a good configuration is therefore essential to obtain the highest possible score.

However, there is a ~~the~~ large number of possible configurations, which makes the tuning of IO500 difficult: a configuration can involve several parameters, and each parameter can have multiple possible values. The combination of these parameters creates a potentially large search space, and finding a good configuration therefore requires evaluating many possible combinations.

Another important difficulty is the cost of an IO500 evaluation. According to the IO500 submission rules [4], each write and create phase must run for at least 300 s~~, using a stonewall timer set to 300 seconds~~. A run that finishes the write phase earlier is considered invalid. This requirement

makes a valid IO500 measurement relatively expensive in terms of cluster time: on the target platform, one valid measurement takes approximately 30 min of exclusive cluster time. When several configurations and repetitions are considered, the total cost becomes very large. For instance, in the parameter space considered in this work, approximately **[TODO:** confirm the exact number**]** 7,840 configurations have to be evaluated. Measuring all of them with three repetitions would require approximately 12 000 h of wall-clock time, which corresponds to more than one year of computation on the target machine. Therefore, An exhaustive search is therefore not practical.

~~Instead, practitioners often rely on their experience and knowledge of the system. Another possible approach is to change one parameter at a time and select the best value before moving to the next parameter. Such approaches can work in some cases, but they may miss interactions between parameters: a configuration that is optimal for one parameter independently is not necessarily optimal when several parameters are considered together.~~

This leads to the main research question of this internship:

> *Can a smarter search method find a near-optimal IO500 configuration using far fewer evaluations, and which search method is most effective for this problem?*

~~The goal is therefore not simply to find a configuration with a high IO500 score. It is also important to understand how many evaluations are required to find this configuration, and how different search strategies behave under a limited evaluation budget.~~

### 1.1. Objectives of the internship

The objective of this internship is to tune IO500 with a guided search instead of an exhaustive one, and to determine which search method should be used in practice. Bayesian Optimization is the main candidate, but it is a family of methods rather than a single one, so a large part of the work consists in comparing its variants, and comparing them against random and exhaustive search.

The work is carried out with IOPS [7], a benchmark orchestration framework developed in the team. IOPS describes a parametric study in a single YAML file and takes care of generating the runs, submitting them to the cluster scheduler, parsing their output and caching the results. Two of its features matter here: the search strategy is a configuration option, so exhaustive, random and Bayesian searches can be compared without changing anything else; and a study can be replayed from the cache, which is what makes the offline comparison possible.

### 1.2. Contributions

The main contributions of this work are the following.

- *A cheap ground-truth methodology* to evaluate search strategies for tuning IO500, based on a set of invalid scores that are much cheaper to generate but still allow the strategies to be compared.
- *A systematic comparison of Bayesian Optimization variants*, which identifies the setup that performs best and shows which of its parameters actually matter.
- *A validation campaign* on the cluster, checking that the configurations selected on the cheap proxy remain the best ones under the official IO500 rules.

### 1.3. Organisation of the report

The remainder of this report is organised as follows. The next section presents the host laboratory and the environment in which the internship took place. The background section then introduces what is needed to read the rest of the report: the IO500 benchmark and its rules, the parameters exposed by parallel file systems, and the principles of Bayesian Optimization. Related work on I/O auto-tuning is reviewed next, followed by a description of the tools, the platform and the experimental setup. The methodology section explains how the ground truth was built and how the search strategies were compared, and the results section reports and analyses the measurements. The report closes with a discussion of the limitations of the study and a conclusion.

## 2. Context of the internship

- **Inria and the Inria Centre at the University of Bordeaux.**
  - Inria: French national research institute for digital science and technology, organised in research centres, each hosting project-teams that are joint with the university.
  - The Inria Centre at the University of Bordeaux: **[TODO:** number of teams and staff, main research axes, location on the Talence campus**]**.
  - Local ecosystem relevant to this work: HPC culture of the lab
- **The host team.**
  - Team TADaaM
  - Position of this internship inside the team's roadmap: storage performance characterisation, I/O benchmarking, and the development of the IOPS framework.
  - Supervision: Luan Teylo.
  - People involved in the broader IO500 effort: **[TODO:** list the collaborators: people we had the meeting**]**.
- **Scientific and technical environment.**
  - **PlaFRIM**: the experimental platform used for all the measurements. **[TODO:** describe: number of nodes, the `bora` partition used here (CPU model, cores per node, memory, interconnect), SLURM as the scheduler**]**.
  - **Storage**: BeeGFS
  - **Software stack**: SLURM, OpenMPI **[TODO:** 4.0.1 was used, confirm**]**, Python 3.10 or later, IOPS, IO500.
- **Objectives and specifications.**
  - Distinguish what was fixed at the start from what evolved during the internship, for instance the addition of the specialist challenge and of the BO diagnostic harness.
- **Organisation of the work.**

– Timeline of the internship: bibliography, tooling, exhaustive campaign, offline study, validation campaign, writing. **[TODO:** a Gantt chart would work well here**]**

– Working method: weekly meetings, Git/GitLab repositories, notebooks and reproducible scripts.

– Deliverables produced: the study repository, the IOPS IO500 configuration, this report, and the intended IOPS technical report.

# Bibliography

## References


[1] Michael A. Heroux, Douglas W. Doerfler, Paul S. Crozier, James M. Willenbring, H. Carter Edwards, Alan Williams, Mahesh Rajan, Eric R. Keiter, Heidi K. Thornquist, and Robert W. Numrich. Improving performance via mini-applications. Technical Report SAND2009-5574, Sandia National Laboratories, 2009. VERIFY: author list and report number.

[2] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015. VERIFY: article number / pages.

[3] IO500 Foundation. The IO500 list. `https://io500.org/`, 2026. Accessed: August 14, 2026.

[4] IO500 Foundation. IO500 submission rules. `https://io500.org/rules/submission`, 2026. Accessed: August 14, 2026.

[5] Frank Schmuck and Roger Haskin. GPFS: A shared-disk file system for large computing clusters. In *Proceedings of the 1st USENIX Conference on File and Storage Technologies (FAST)*, pages 231–244, 2002. VERIFY: page numbers.

[6] Philip Schwan. Lustre: Building a file system for 1000-node clusters. In *Proceedings of the 2003 Linux Symposium*, 2003. VERIFY: page numbers.

[7] TADaaM, Inria. IOPS: Integrated orchestration for parametric studies. `https://iops-benchmark.com`, 2026. Source repository: `https://gitlab.inria.fr/hpc_io/iops`. Accessed: August 14, 2026.

[8] The IOR contributors. IOR and mdtest: Parallel filesystem I/O benchmarks. `https://github.com/hpc/ior`, 2026. Accessed: August 14, 2026.

[9] The IOR contributors. mdtest: Metadata benchmark for parallel file systems. `https://github.com/hpc/ior`, 2026. Distributed with the IOR suite. Accessed: August 14, 2026.

[10] The scikit-optimize contributors. scikit-optimize: Sequential model-based optimization in Python. `https://scikit-optimize.github.io/`, 2026. Version TO-COMPLETE.

[11] ThinkParQ GmbH. BeeGFS: The parallel cluster file system. `https://www.beegfs.io/`, 2026. Accessed: August 14, 2026.

[12] TO-COMPLETE. To-complete: 2022 I/O performance study. `https://inria.hal.science/hal-03753813/`, 2022.

[13] TO-COMPLETE. To-complete: IO500 tuning on CLAIX23. In *IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*, 2025.

[14] TO-COMPLETE. To-complete: IO500 benchmark reference paper, 2026.

[15] TOP500.org. The TOP500 list. `https://www.top500.org/`, 2026. Accessed: August 14, 2026.